ARGONAUT

AI-Powered Job-Verification for India

PRODUCT & TECHNOLOGY OVERVIEW

Argonaut is a fraud-intelligence platform that verifies LinkedIn and WhatsApp job posts before a candidate replies. A job seeker pastes a suspicious post and receives a **0–100 trust score** in seconds, with every red flag, extracted detail, and recommendation laid out. The verification engine — **Argus** — runs **entirely inside the user's web browser**: no account, no API key, no server, and nothing the user pastes ever leaves their device.

EXHIBIT I

The Problem It Solves

Fake job posts are one of the most common online frauds in India. Scammers copy-paste the same lures across hundreds of WhatsApp groups and LinkedIn feeds — registration fees, fake offer letters, ‘security deposits’, and identity-theft forms that harvest Aadhaar, PAN, and bank details. The people who lose money are usually the ones who never doubted the post. Argonaut gives every job seeker a fast, free second opinion at the exact moment they need it.

EXHIBIT II

Who It Helps & How

- **Job seekers** — paste any post and instantly see whether it is legitimate, engagement bait, suspicious, or an outright scam — with the reasons why.
- **Freshers & students** — the highest-risk group, protected by a plain-English verdict and an interactive Learning Hub that trains a sharper eye.
- **Placement cells & HR teams** — a batch pipeline screens hundreds of posts at once and exports a structured hazard report.
- **The whole community** — a shared scam blocklist (fraud phone numbers, UPI handles, and domains) is cross-checked on every scan, so one person's report protects the next.

EXHIBIT III

What the App Does

Area	What it does
Scanner	Paste post text or a screenshot → trust score 0–100, verdict, red/green flags, extracted job details, and an evidence trail.
Duplicate check	Posts are fingerprinted on-device; known or lightly-edited copies return instantly from cache. Screenshots matched by perceptual hashing.

Telemetry	Live dashboard: scan-volume trend, verdict distribution, regional activity, average trust score, and curated threat advisories.
Intelligence	Company trust ledger, verified-employer registry, community scam blocklist, personal applicant vault, and full audit trail.
Batch	B2B / placement-cell pipeline: paste many posts or upload a CSV, then export a structured hazard report.
Learn	Five micro-modules plus a ‘Spot-the-Scam’ simulator that builds cognitive defenses.
Panic Button	Instant on-device check of an interview or meeting link for typosquats, IP links, brand impersonation, and shorteners.

EXHIBIT IV

The Technology & Tools

A common question is which language the model uses. Argonaut uses **two**, each for what it does best: **Python** to *train* the model, and **JavaScript** to *run* it on the user's device.

Trained in Python, runs in the browser

The model is trained offline using **Python and scikit-learn** — the standard machine-learning toolkit. The training compares five algorithms, tunes the best one, and **learns** which words and patterns signal a scam from labelled examples. The finished, learned weights are then exported into a small file that the browser runs directly in **JavaScript**. This is exactly how on-device ML works on phones (Google and Apple do the same): heavy training happens once on a computer; lightweight prediction runs on the device.

Why not run Python live on a server? That would mean hosting cost, an API, and the user's data leaving their device. By shipping only the trained weights and predicting in the browser, Argonaut keeps the whole promise: **a genuinely data-trained model, with no key, no cost, and nothing leaving the user's device.** (The browser's prediction was verified to match Python's to three decimal places.)

Layer	Tool / Technology	Purpose
Model training	Python + scikit-learn	Trains, tunes & evaluates the fraud model offline; exports learned weights.
Verdict engine	JavaScript (js/model.js)	Runs the trained model on-device and blends it with the rule engine.
Screenshot reading	Tesseract.js (OCR)	Reads text out of uploaded images in the browser, then feeds it to the model — the image is never uploaded.
App logic	Vanilla JavaScript	Scanner, dashboards, routing — no framework, no build step.
Landing visuals	three.js + GSAP	The particle ‘Voyage’ animation (shipped locally).
Design	HTML + CSS	Museum-style parchment & bronze interface.
Storage	Browser localStorage	Scans, ledger, vault — kept only on the device.

Hosting	GitHub Pages (static)	Free, no backend; also works fully offline by file.

EXHIBIT V

How the Argus Model Generates a Result

When a user pastes a post and clicks Analyze, the model runs a six-step pipeline — all in the browser, in under two seconds:

1. Read the post apart (extraction)

Pattern-matching pulls out the company, role, location, salary, and any phone numbers, UPI handles, emails, and links — e.g. a 10-digit number starting 6–9 is an Indian phone.

2. Weigh danger words vs. safe words (lexicons)

The text is checked against built-in lists of scam phrases, each with a weight: ‘registration fee’, ‘security deposit’ (money fraud); ‘comment YES’, ‘DM me’ (bait); ‘Aadhaar / PAN’ (identity theft). Safe words push the other way: ‘careers.company.com’, ‘interview rounds’, ‘notice period’.

3. Six specialist detectors vote

Six mini-checkers each judge one angle — money fraud, data harvesting, engagement bait, unrealistic pay, legitimacy, and spammy formatting — and raise the red and green flags shown in the result.

4. Score it with the trained model

A **Logistic-Regression model trained in Python** reads the text and outputs a fraud probability — using weights it **learned from labelled examples**, not hand-written rules. This is blended with the rule engine (60/40 by default) for robustness. The result is a fraud probability between 0 and 1, then: **Trust Score = (1 – fraud probability) x 100.**

5. Cross-check registry & community

A recognised employer nudges trust up; a phone or UPI already on the community scam blocklist slams the score down.

6. Decide the verdict & write the summary

Based on the score and which flags fired, the post is labelled **Legitimate, Engagement Bait, Suspicious, or Actual Scam**, with a plain-English summary and recommendation.

EXHIBIT VI

How the Model Is Trained & Improved

The model is not hand-written — it is **trained**. A Python pipeline (scikit-learn) takes a set of labelled job posts (each marked scam or legit), compares five machine-learning algorithms, tunes the best one, and measures its accuracy on posts it has never seen. The learned weights are then exported to run in the browser.

- **Five models compared** — Naive Bayes, Logistic Regression, Linear SVM, Random Forest, and Gradient Boosting — scored by cross-validation.
- **Tuned** — a grid search finds the best settings for the chosen model.
- **Measured** — accuracy, precision, recall, and F1 on a held-out test set.

- **Improvable in one command** — add real labelled posts to the dataset file and re-run training; the app picks up the new model on its next load.

The current build ships with a realistic **seed dataset** to prove the pipeline. Because template data is easy to separate, its test scores read near-perfect — that is expected, and not a claim of real-world accuracy. Feeding in real, varied, labelled posts is what turns it into a production-strength model, and the pipeline is built for exactly that.

EXHIBIT VII

A Worked Example

Post: *"URGENT! Comment YES, WhatsApp 98xxxx10, pay Rs.500 registration, limited seats!!"*

- **Extraction:** finds the phone number and a 'Rs.500' amount.
- **Lexicons fire:** 'registration' (money, heavy), 'comment YES' (bait), 'limited seats' (urgency), WhatsApp + phone (off-platform), '!!' and CAPS (spam).
- **Formula:** fraud signals stack up with nothing pushing back — fraud probability ≈ 0.99.
- **Result:** Trust Score = (1 – 0.99) x 100 ≈ **1 / 100** — verdict **ACTUAL SCAM**, with four red flags listed.

By contrast, a genuine post — e.g. a Razorpay engineering role linking to careers.razorpay.com with clear interview rounds — scores **97 / 100, Legitimate**. (The engine never claims a perfect 100: a probabilistic advisory always leaves room to verify independently.)

EXHIBIT VIII

How the Result Is Displayed

The model returns a structured result — score, verdict, flags, and extracted details. The interface then draws the result card: the trust gauge, the verdict badge, the red-flag list, and the extracted-details grid. The model produces the data; the design displays it. Every scan is also written to an on-device audit trail, so any verdict can be explained step-by-step.

EXHIBIT IX

Privacy & Compliance

- **Fully on-device:** every scan is analysed in the browser — no server, no third party, no upload. This makes the privacy promise literally true.
- **DPDP Act 2023:** itemized, withdrawable consent on first run; scans, fingerprints, the vault, and audit trails live only in the browser and can be exported or erased anytime.
- **Advisory, not accusation:** trust scores are probabilistic safety advisories, not legal determinations. Every result carries a reporting path (cybercrime.gov.in).

EXHIBIT X

Why This Approach Is Strong

- **Genuinely data-trained** — a real machine-learning model (Python / scikit-learn), not just hand-written rules, that improves as more labelled data is added.
- **Works instantly for everyone** — no sign-up, no API key, no waiting.

- **Free and unlimited** — zero cost per scan, because there is no paid service behind it.
- **Private by design** — the strongest possible answer to ‘what happens to my data?’
- **Fully owned** — the model is Argonaut's own code and weights, explainable and tunable, not a rented third-party service.
- **Fast & offline-capable** — runs even on a low-end phone with no connection.